

Labor zur Vorlesung Kommunikationstechnik

Laborübung CRC

Prof. Dr. Dirk Staehle

Die Abgabe erfolgt durch Hochladen der bearbeiteten Word-Datei in Moodle.

Bearbeitung in Zweier-Teams

Team-Mitglied 1:

Team-Mitglied 2:

1 Einleitung

In dieser Aufgabe wollen wir unseren bekannten RFC über UART von einem Micropython Device (Arduino NANO RP2040, Arduino ESP32, Raspberry Pi Pico 2 W) zu einem zweiten Micropython Device übertragen. Damit wir Fehler bekommen, schalten wir einen Arduino Mega dazwischen. Übertragungsfehler sollen mittels einer CRC-Prüfsumme erkannt werden und durch Wiederholungsübertragungen nach einem einfachen Stop-and-Wait-Verfahren behoben werden.

2 Vorbereitung

2.1 Thonny IDE

Für die Programmierung der Devices empfiehlt sich die Thonny IDE (<https://thonny.org/>). Alternativ dazu kann auch die minimalistische IDE „Arduino Lab for MicroPython“ (<https://labs.arduino.cc/en/labs/micropython>) oder ein MicroPython PlugIn z.B. für PyCharm oder Visual Studio Code verwendet werden.

2.2 Installieren der MicroPython Firmware

Mit Thonny können Sie die MicroPython Firmware direkt installieren (Run→Configure Interpreter→Interpreter→Install or Update MicroPython (esptool)(UF2). Hier können Sie verschiedene Firmware Varianten auswählen. Alternativ dazu kann die Firmware auch geladen werden und einfach in den Flash-Speicher kopiert werden (unter Windows via Explorer).

In beiden Fällen muss das Installation das Device vor der Installation in den Bootloader Mode versetzt werden. Zuverlässig geht das durch:

- bei einem NANO RP2040: Verbinden der REC und GND Pins mit einem Jumper Kabel, anschließend Reset Button drücken und Kabel entfernen → unter Windows sollte sich ein Laufwerk im Explorer zeigen. Anleitung ohne Thonny hier:

<https://www.codemotion.com/magazine/backend/getting-started-with-micropython-on-arduino-nano-rp2040-connect/>

- bei einem NANO ESP32: Verbinden der B1 und GND Pins mit einem Jumper Kabel, anschließend Reset Button drücken und Kabel entfernen → unter Windows sollte sich ein Laufwerk im Explorer zeigen. Anleitung ohne Thonny hier:

<https://support.arduino.cc/hc/en-us/articles/9810414060188-Reset-the-Arduino-bootloader-on-the-Nano-ESP32>

- bei einem Raspberry Pi Pico: BOOTSEL Button gedrückt halten, während das Board an USB angeschlossen wird

2.3 Arduino Mega

Den Arduino Mega programmieren wir in der Arduino eigenen Programmiersprache, die auf C basiert. Als die nutzen wir die Arduino IDE

<https://www.arduino.cc/en/software>

3 Verkabelung

Wir verbinden die TX/RX Pins (UART0) der Nanos mit den RX1/TX1 (Serial1) bzw. TX2/RX2 Pins (Serial2) des Megs.

4 Versuchsaufbau

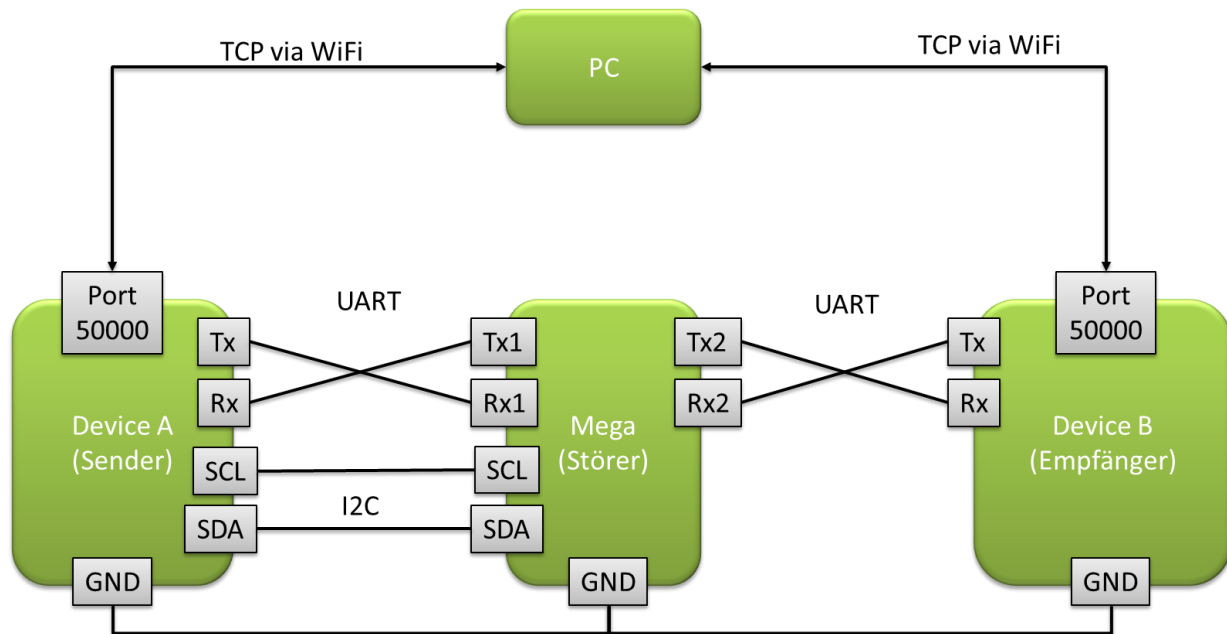
In unserem Versuch wollen wir wie oben beschrieben, eine UART Übertragung mit einem Stop-and-Wait Protokoll durchführen. Die Übertragung soll von Device A zu Device B erfolgen, die beide mit MicroPython programmiert werden sollen. Damit wir die Bitfehlerrate kontrollieren können, läuft die UART-Übertragung über einen Arduino Mega, der Bits mit einer vorgegebenen Bitfehlerwahrscheinlichkeit kippt.

Neben der grundsätzlichen Aufgabe, den RFC über die fehlerbehaftete UART-Strecke zu übertragen, wollen wir auch untersuchen, welche Frame-Länge und welches Generatorpolynom für die Übertragung am Besten geeignet ist. Um Versuchsreihen automatisiert durchführen zu können (und weil wir in Kommunikationstechnik damit experimentieren wollen), verbinden wir die beiden Devices A und B via WLAN mit dem PC, der den beiden Devices dann die jeweiligen Parameter für einen Versuch (Frame-Länge, Generatorpolynom, Anzahl Bits für Sequenznummer, Anzahl zu übertragender Frames, Baudrate der seriellen Verbindung und BER) mitteilt. Device A teilt dem Mega über I2C die BER mit und beginnt dann mit der Übertragung. Device B empfängt die Daten und führt eine statistische Auswertung durch, d.h. es werden erfasst

- die Anzahl korrekter Übertragungen
- die Anzahl erkannter Übertragungsfehler (Übertragungswiederholungen)
- die Anzahl unerkannter Übertragungsfehler (keine Fehlererkennung, aber Vergleich mit Original zeigt keine Übereinstimmung)
- die Anzahl fehlerhafter Sequenznummern, hier unterscheiden wir
 - Wiederholung des zuletzt bestätigten Frames (als Folge eines ACK Fehlers)
 - Vollkommen falsche Sequenznummer (als Folge eines nicht erkannten Übertragungsfehlers in der Sequenznummer)

Nach der Bestätigung des letzten Frames (und nach einer kurzen Wartephase, in der sichergestellt wird, dass das ACK angekommen ist und keine Wiederholungsübertragung mehr eintrifft) sendet Device B die Daten an den PC, der dann einen neuen Versuch starten kann.

In der folgenden Grafik sehen Sie den Versuchsaufbau.



Um den Versuch zu implementieren, gehen wir schrittweise vor:

1. Fehlerfreie UART Datenübertragung
2. Einbauen von Bitfehlern mit fester Bitfehlerwahrscheinlichkeit
3. Konfigurieren der Bitfehlerwahrscheinlichkeit über I2C
4. Fehlererkennung mit CRC
5. Stop-and-Wait-Protokoll (Sequenznummern, ACKs mit CRC, Wiederholungsübertragung)
6. Statistische Erfassung
7. Übertragung von Konfigurationsparametern vom PC
8. Durchführen von Versuchsreihen mit graphischer Darstellung

4.1 UART Datenübertragung

Im ersten Schritt wollen wir Daten (den RFC) über UART von Device A zu Device B übertragen und auch eine Ausgabe auf dem Mega erzeugen.

UART mit MicroPython: <https://docs.micropython.org/en/latest/library/machine.UART.html>

Arduino: Nutzen Sie das Beispiel „MultiSerial“ als Startpunkt für die Implementierung auf dem Mega. In unserem Beispiel kommen die Daten auf Serial1 bzw. Serial2 an und wir wollen die Daten auf Serial2 bzw. Serial1 weiterleiten und zusätzlich eine Ausgabe auf Serial produzieren (Anschauen mit Lupe).

Achten Sie darauf, auf allen Devices die gleiche Baudrate einzustellen.

4.2 Bitfehler

Implementieren Sie eine Funktion auf dem Mega, der nach einer vorgegebenen Bitfehlerwahrscheinlichkeit in den empfangenen Bytes Bits umkippt und die potenziell fehlerhaften Bytes weiterleitet.

Übertragen Sie einen Teil des RFCs und prüfen Sie, ob die Bitfehlerwahrscheinlichkeit stimmt. Laden Sie dazu den RFC in beide Devices.

Tipps:

- Arduino unterstützt nur `random(min,max)`
- geben Sie die Bitfehlerwahrscheinlichkeit als Vielfaches von $1/65536$ an und verwenden Sie `random(65536)`
- bestimmen Sie für jedes Bit einzeln, ob es gekippt wird
- Bit i eines Bytes b können Sie kippen durch $b^i = (1 < i)$

4.3 Konfigurieren der Bitfehlerwahrscheinlichkeit

Verbinden Sie SCL und SDA von Device A mit SCL und SDA auf dem Mega. Verwenden Sie Device A als Master und den Mega als Slave. Informationen zu I2C in MicroPython finden Sie hier (<https://docs.micropython.org/en/latest/library/machine.I2C.html>). Verwenden Sie hier die Befehle „`writeto`“ und „`readfrom`“, um Daten zu senden und zu empfangen. Unter Arduino nutzen Sie die Wire-Library und starten mit dem Sketch `slave-receiver`. Mehr zu I2C folgt im Laufe der Vorlesung.

Übertragen Sie neben der Bitfehlerrate auch noch die Baudrate der seriellen Verbindung.

4.4 Absicherung mit CRC

Implementieren Sie in Python/MicroPython eine Fehlererkennung mit CRC. Verwenden Sie dazu als Basis den CRC-Algorithmus aus der Vorlesung. Unterstützen Sie CRCs mit 4, 6, 8 und 16 Bit. Nutzen Sie dazu bekannte Generatorpolynome (<https://reveng.sourceforge.io/crc-catalogue/all.htm>). Übertragen Sie die CRC-Prüfsumme immer als ganze Bytes – auch wenn tatsächlich weniger Bits übertragen werden – das macht die Implementierung deutlich einfacher.

Beginnen Sie mit einer kleinen Nutzwortlänge. Testen Sie zunächst lokal, ob die Fehlererkennung für verschiedene Bitfehlerwahrscheinlichkeiten zuverlässig funktioniert.

4.5 Stop-and-Wait

Damit uns die Fehlererkennung auch etwas bringt, implementieren wir ein Stop-and-Wait-Protokoll, d.h. wir senden nach erfolgreichem Empfang ein ACK und wiederholen die Übertragung, wenn nach einem Timeout kein ACK eingetroffen ist. Wir benötigen in unseren Datenframes jetzt neben den CRC-Bits jetzt auch noch eine Sequenznummer. Auch in den ACK-Frame schreiben wir eine Sequenznummer und sollten uns überlegen, ob wir den ACK-Frame auch mit Prüfbits versehen.

4.6 Dateien-Übertragung und statistische Erfassung

Wir wollen jetzt unseren RFC 2324 (oder zumindest Teile davon) übertragen und im zweiten Schritt die Performance (Datenintegrität (unerkannte Fehler), Übertragungsdauer, Overhead (Bytes im Header), Anzahl Wiederholungsübertragungen) in Abhängigkeit der Bitfehlerwahrscheinlichkeit, der Framegröße und der Anzahl Prüfbits betrachten. Wir bestimmen dazu in Device B

- die Anzahl korrekter Übertragungen
- die Anzahl erkannter Übertragungsfehler (Übertragungswiederholungen)
- die Anzahl unerkannter Übertragungsfehler (keine Fehlererkennung, aber Vergleich mit Original zeigt keine Übereinstimmung)
- die Anzahl fehlerhafter Sequenznummern, hier unterscheiden wir
 - Wiederholung des zuletzt bestätigten Frames (als Folge eines ACK Fehlers)
 - Vollkommen falsche Sequenznummer (als Folge eines nicht erkannten Übertragungsfehlers in der Sequenznummer)

4.7 Konfiguration via WiFi

Um eine Parameter-Studie effizient durchführen zu können, wollen wir eine Simulation von einem PC aus anstoßen und die Ergebnisse auch wieder einsammeln können. Dazu verbinden sich die beiden Devices und der PC in einem gemeinsamen WLAN. Die beiden Devices öffnen jeweils einen Port, auf den sich der PC verbindet. Zum Starten einer Simulation sendet der PC via TCP z.B. im JSON-Format die Simulationsparameter an die Devices A und B. Device A (der Sender), startet die Simulation und nach Ende der Simulation sendet Device B (der Empfänger) die Simulationsergebnisse an den PC. Am Ende der Simulationsstudie werden auf dem PC die entsprechenden Grafiken dargestellt.

Hinweise zur Verwendung von WLAN in MicroPython:

- <https://docs.micropython.org/en/latest/library/network.WLAN.html>

4.8 Parameterstudien mit graphischer Darstellung

Das Ziel unserer Parameterstudie ist, eine geeignete Kombination aus CRC Polynom (Anzahl Parity-Bits) und Framelänge für eine Bitfehlerrate von 0,0005 (0,05%) zu bestimmen. Dabei sollen bei 5 Übertragungen des RFC höchstens 3 nicht erkannte Frame-Fehler auftreten. Zusätzlich soll die die Anzahl der benötigten Übertragungen minimiert werden.